



Calibration et Simulation

TP3 : Modèle de Vasicek

STEPHAN Corentin

Table des matières

I - Modèle de Vasicek	3
II. Calibration de la courbe des taux (Yield Curve).....	5
III. Dates historiques	7
3.1 - Régression par Levenberg-Marquardt.....	8
3.2 - Régression par la méthode analytique	9
3.1	

I - Modèle de Vasicek

Code :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import cm

gamma = 0.25
eta = 0.25*0.03
sigma = 0.02
t = 0
r0 = 0.01

# Partie I
def Vasicek (r0):
    T = np.arange(0.1, 30, 1)
    P = np.zeros(len(T))
    A = np.zeros(len(T))
    B = np.zeros(len(T))
    Y1 = np.zeros(len(T))
    Y2 = np.zeros(len(T))

    for i in range(len(T)):
        td = T[i] - t
        B[i] = (1 - np.exp(-gamma * td)) / gamma
        A[i] = (B[i] - td) * ((eta * gamma - (sigma**2) / 2) / gamma**2) - (((sigma**2) * B[i]**2) / (4 * gamma))
        P[i] = np.exp(A[i] - (r0 * B[i]))
        Y1[i] = -np.log(P[i]) / td
        Y2[i] = -(A[i] - r0 * B[i]) / td
    plt.plot(T, Y1, label=f"r0 = {r0}")

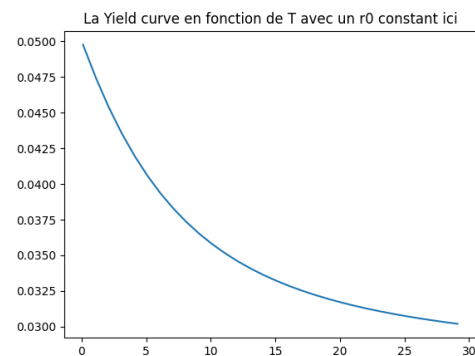
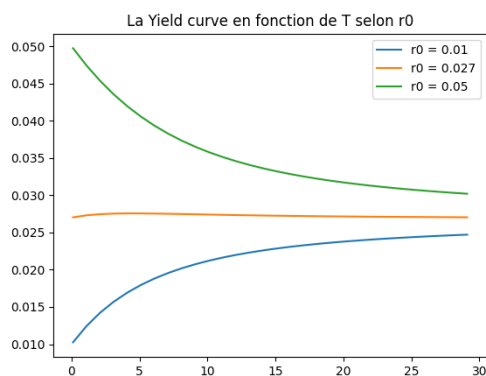
def SimuVasicek():
    r0 = [0.01, 0.027, 0.05]
    for i in range(3):
        Vasicek(r0[i])
    plt.legend()
    plt.title("La Yield curve en fonction de T selon r0")
    plt.show()

    Vasicek(0.05)
    plt.title("La Yield curve en fonction de T avec un r0 constant ici")
    plt.show()

    limite_th = eta/gamma - 0.5 * (sigma/gamma)**2
    print("La limite théorique de la formule :  $(\eta/\gamma) - 1/2 * (\sigma/\gamma)^2$  est", limite_th)

SimuVasicek()
```

Graphiques :



Interprétation :

Pour travailler sur la modélisation des taux d'intérêt, nous utilisons le modèle de Vasicek. Les paramètres peuvent varier lors de calibration cependant les courbes tendent à revenir vers une moyenne au fil du temps comme le montre le graphique 1. Ce principe se nomme : "principe de réversion à la moyenne". En effet, les taux d'intérêt suivent un processus stochastique qui a pour particularité de revenir vers une moyenne sur le long terme.

- η (vitesse de réversion) : plus il est élevé, plus les chocs de marché sont absorbés rapidement
- σ (volatilité) : plus il est élevé, plus il y aura des fluctuations aléatoires autour de la moyenne

Nous pouvons estimer vers quelle valeur les courbes ont tendance à aller avec la formule suivante :

$$\lim_{T \rightarrow 0} Y(0, T) = r_0, \quad \lim_{T \rightarrow \infty} Y(0, T) = \frac{\eta}{\gamma} - \frac{1}{2} \left(\frac{\sigma}{\gamma} \right)^2$$

Ainsi, nous pouvons calculer cette valeur et avoir :

La limite théorique de la formule : $(\eta/\gamma) - 1/2 * (\sigma/\gamma)^2$ est 0.02679999999999997

Code :

```
def Yield(r0, T, t):
    B = (1 - np.exp(-gamma * (T - t))) / gamma
    A = (B - T) * ((eta * gamma - (sigma**2) / 2) / gamma**2) - (((sigma**2) * B**2) / (4 * gamma))
    P = np.exp(A - (r0 * B))
    Y = -np.log(P) / (T - t)
    return Y

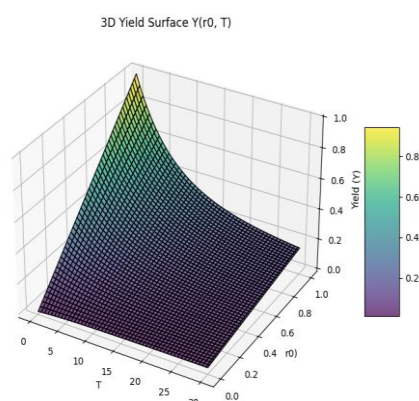
def Surface():
    T = np.linspace(0.1, 30, 100)
    r0 = np.linspace(0, 1, 50)
    T_grid, r0_grid = np.meshgrid(T, r0)

    Y_grid = Yield(r0_grid, T_grid, t)

    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')
    surf = ax.plot_surface(T_grid, r0_grid, Y_grid, cmap=cm.viridis, edgecolor='k', alpha=0.7)
    fig.colorbar(surf, shrink=0.5, aspect=5)

    ax.set_xlabel('T')
    ax.set_ylabel('r0')
    ax.set_zlabel('Yield (Y)')
    ax.set_title("3D Yield Surface Y(r0, T)")
    plt.show()

Surface()
```

Graphique :

II. Calibration de la courbe des taux (Yield Curve)

Code :

```
# Partie II et III
def Levenberg_Marquart(t, yield_m, r0):
    T = np.linspace(3, 30, 10)
    epsilon = 10**(-9)
    beta = [1, 1, 1]
    lambdas = 0.01
    d = [1, 1, 1]
    Res = np.zeros(10)
    J = np.array([np.zeros(3) for i in range(10)])
    tab_A = []
    tab_B = []

    compteur = 0

    while np.linalg.norm(d) > epsilon:
        compteur += 1
        for i in range(10):
            #Calcul explicite du TD
            B = (1 - np.exp(-beta[2] * (T[i] - t))) / beta[2]
            A = (B - (T[i] - t)) * ((beta[0] * beta[2] - (beta[1] / 2)) / beta[2]**2) - ((beta[1] * (B**2)) / (4 * beta[2]))
            #Calcul des dérivées explicite de A et de B
            DB = ((T[i] - t) * np.exp(-(T[i] - t) * beta[2]) - B) / beta[2]
            DA = (
                beta[0] * (DB * beta[2] - B) + (T[i] - t) * beta[0]
                - beta[1] / 2 * (DB - 2 * B / beta[2])
                - (T[i] - t) * beta[1] / beta[2]
                - beta[1] * B / 4 * (2 * beta[2] * DB - B)
            ) / beta[2]**2
            #On stock les dérivées dans une liste à chaque itérations
            tab_A.append(A)
            tab_B.append(B)

            #Calcul des de J i explicitement
            J[i][0] = (B - (T[i] - t)) / (T[i] - t) / beta[0]
            J[i][1] = -((B - (T[i] - t)) / (2 * beta[2]) + (B**2) / 4) / ((T[i] - t) * beta[2])
            J[i][2] = (DA - r0 * DB) / (T[i] - t)
            #Calcul de Y données dans la première partie
            Yt = -(A - r0 * B) / (T[i] - t)
            #On calcul le résidu qui est la différence en le yield calculé et la valeur marché
            Res[i] = yield_m[i] - Yt

        #On calcul la valeur de d_k
        d = np.dot(-np.linalg.inv(np.dot(J.transpose(), J) + lambdas * np.identity(3)), np.dot(J.transpose(), Res))
        beta = beta + d.transpose()

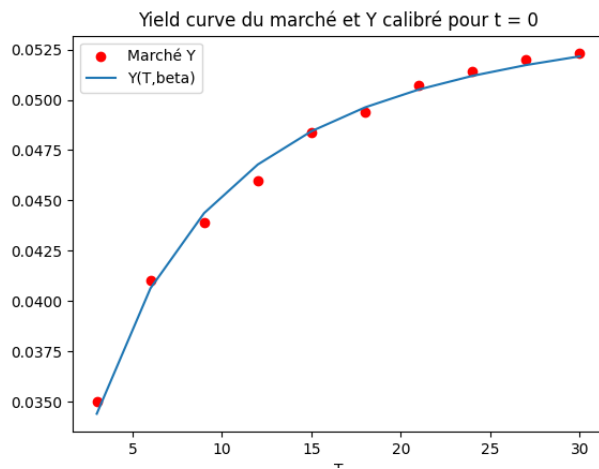
    print("Nombre d'iterations:", compteur)
    print("beta*1 (eta) =", beta[0])
    print("beta*2 (sigma) =", beta[1])
    print("beta*3 (gamma) =", beta[2])
```

```
Y = []
for i in range(1, 11):
    B = (1 - np.exp(-beta[2] * (T[i - 1] - t))) / beta[2]
    A = (B - (T[i - 1] - t)) * ((beta[0] * beta[2] - (beta[1] / 2)) / beta[2]**2) - ((beta[1] * (B**2)) / (4 * beta[2]))
    Y.append(-(A - r0 * B) / (T[i - 1] - t))

plt.scatter(T, yield_m, label="Marché Y", color="red")
plt.plot(T, Y, label="Y(T,beta)")
plt.legend()
plt.xlabel('T')
plt.ylabel('Y')
plt.title("Yield curve du marché et Y calibré pour t = " + str(t))
plt.show()

i = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
T = [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
yield_market = [0.035, 0.041, 0.0439, 0.046, 0.0484, 0.0494, 0.0507, 0.0514, 0.052, 0.0523]
r0 = 0.023
Levenberg_Marquart(0, yield_market, r0)

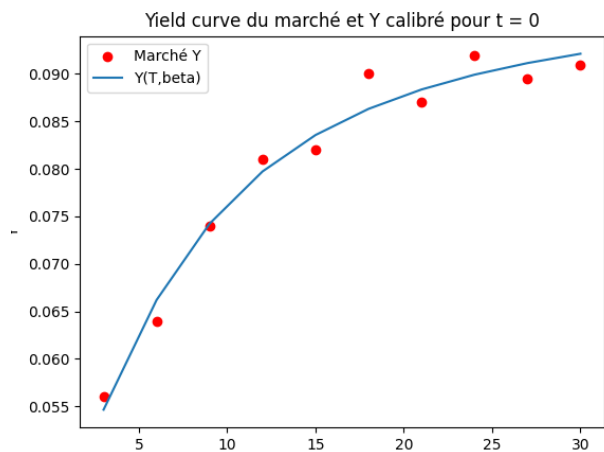
i = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
T = [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
yield_m = [0.056, 0.064, 0.074, 0.081, 0.082, 0.09, 0.087, 0.092, 0.0895, 0.091]
r0 = 0.04
Levenberg_Marquart(0, yield_m, r0)
```

Graphiques :

```

Nombre d'iterations: 5121
beta^1 (eta) = 0.015375669687660659
beta^2 (sigma^2) = 0.001418209435075014
beta^3 (gamma) = 0.2153970693838513

```



```

Nombre d'iterations: 593
beta^1 (eta) = 0.020870766244188994
beta^2 (sigma^2) = -0.003968341655053278
beta^3 (gamma) = 0.27743603263271055

```

Interprétation :

Pour tracer les courbes de taux, nous allons utiliser l'algorithme de Levenberg-Marquardt. A partir de cet algorithme et en sachant que les courbes suivent le modèle de Vasicek, nous estimons les paramètres η , σ^2 et γ . Nous pouvons comparer les valeurs estimées pour $t=0$ et $t=1$. Les paramètres η , σ^2 et γ sont plus stables à $t=0$ qu'à $t=1$. Au fur et à mesure que l'horizon de temps s'allonge, les incertitudes augmentent et donnent donc des estimations moins précises. A l'aide des graphiques, nous constatons que les points du marché s'éloignent de plus en plus de la courbe théorique à mesure que la maturité T augmente.

Ces calibrations permettent donc de prédire les courbes de taux pour plusieurs échéances (ici $t=0$ et $t=1$). Avec le principe de réversion, nous pouvons évaluer des produits sur le long terme sous nécessité d'avoir les paramètres η et σ^2 .

III. Dates historiques

Code :

```
def DateHisto():

    #Liste de paramètres
    r0 = 0.023
    N=1000
    T=5
    dt=T/N
    eta=0.6
    sigma=0.08
    gamma=4
    t = [i/N for i in range(N+1)]
    beta = [1, 1]
    d = [1, 1]
    epsilon = 10**(-4)
    J = np.array([np.zeros(2) for i in range(N)])
    r_f = []
    beta = [1, 1]
    d = [1, 1]
    lam = 0.1
    tab_r = [r0]
    epsilon = 10**(-4)
    var = np.zeros(N)

    # Calcul de r explicite
    for i in range(N):
        tab_r.append(tab_r[-1] * np.exp(- gamma * dt) + (eta/gamma)*(1-np.exp(-gamma * dt)) + sigma * np.sqrt( (1-np.exp(-2*gamma*dt))/ 2*gamma) * np.random.randn())

    plt.plot(t, tab_r)
    plt.title("Simulation de rt du marché suivant Vasicek en fonction de t")
    plt.xlabel('t')
    plt.ylabel('r')
    plt.show()
    x = [tab_r[i] for i in range(N-1)]
    y = [tab_r[i+1] for i in range(N-1)]
    plt.scatter(x, y)
    plt.xlabel('r_i')
    plt.ylabel('r_(i+1)')
    plt.title("Simulation de la fonction y=f(x)")
    plt.show()

    # Déterminons Beta1 et Beta2
    while np.linalg.norm(d) > epsilon:
        r = [0.023]
        for i in range(N-1):
            # Calcul de r explicite
            r.append(r[-1]*np.exp(-gamma*dt)+eta/gamma*(1-np.exp(-gamma*dt))+np.sqrt((sigma**2)/(2*gamma)*(1-np.exp(-2*gamma*dt)))*np.random.randn())
            J[i][0] = -r[i]
            J[i][1] = -1
            var[i] = r[-1]-(beta[0]*r[-2]+beta[1])
        # Calcul de d de façon explicite
        d = np.dot(-np.linalg.inv(np.dot(J.transpose(), J) + lam*np.identity(2)), np.dot(J.transpose(), var))
        beta = beta + d.transpose()
    a = beta[0]
    b = beta[1]
    print("Beta1 = "+str(beta[0]))
    print("Beta2 = "+str(beta[1]))

    # Calcul de D explicite
    D_carre = np.var(var)
    D = np.sqrt(np.var(var))
    # Calcul de gamma, eta et sigma explicitement
    gamma = -np.log(a)/dt
    eta = gamma* (b/(1-a))
    sigma = D * np.sqrt((-2*np.log(a))/(dt*(1-a**2)))

    print("variance D² = "+str(D_carre))
    print("D = "+str(D))
    print('Gamma = '+str(gamma))
    print('Sigma = '+str(sigma))
    print('Eta = '+str(eta))

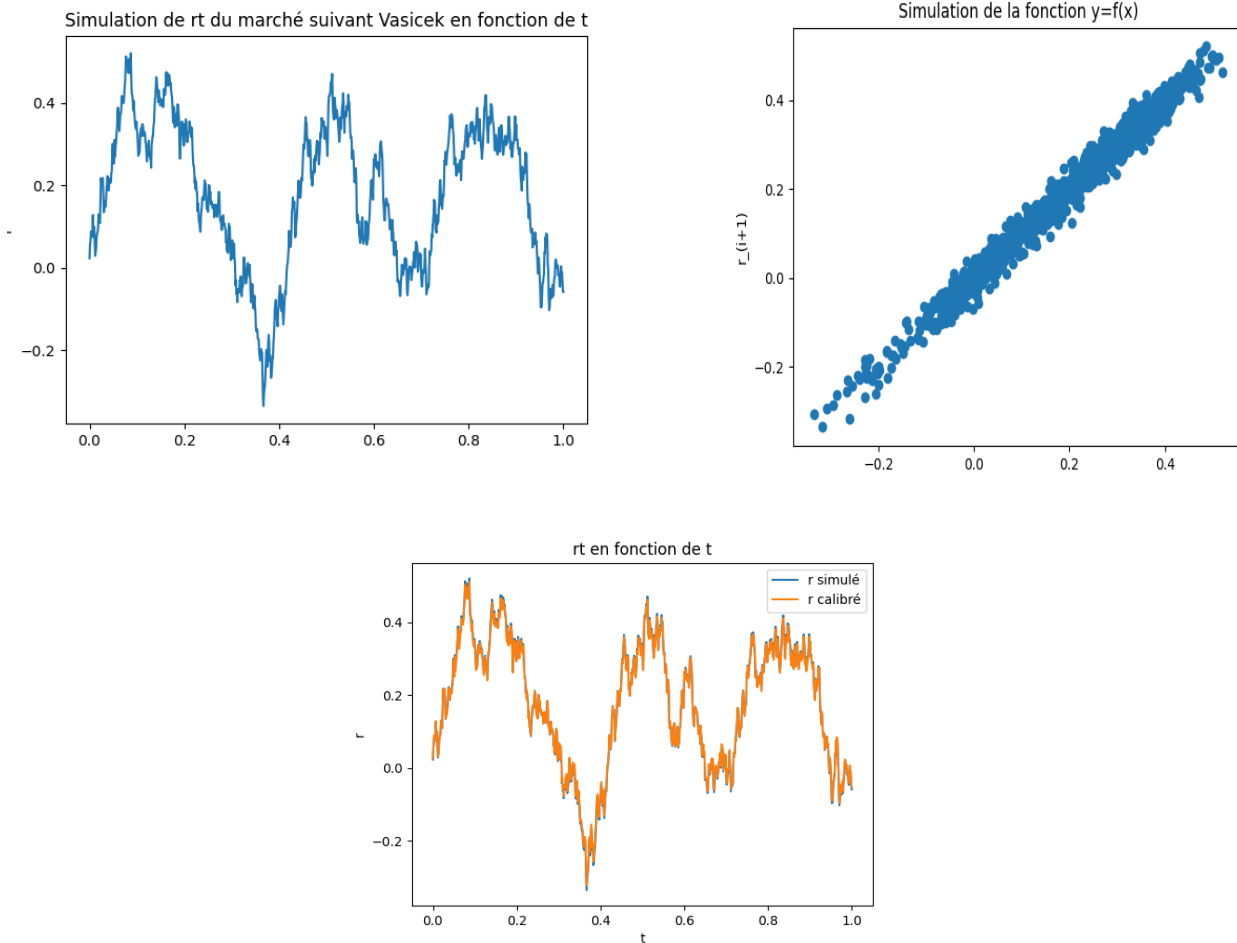
    #calcul r calibré
    for i in range(N+1):
        r_f.append(beta[0]*tab_r[i] + beta[1])

    plt.plot(t, tab_r, label = 'r simulé')
    plt.plot(t, r_f, label = 'r calibré')
    plt.legend()
    plt.xlabel('t')
    plt.ylabel('r')
    plt.title("rt en fonction de t")
    plt.show()

    plt.scatter(x, y, label = 'r simulé')
    plt.plot(tab_r, r_f, label = 'r calibré', color="orange")
    plt.legend()
    plt.xlabel('r_i')
    plt.ylabel('r_(i+1)')
    plt.title("fonction y=f(x)")
    plt.show()

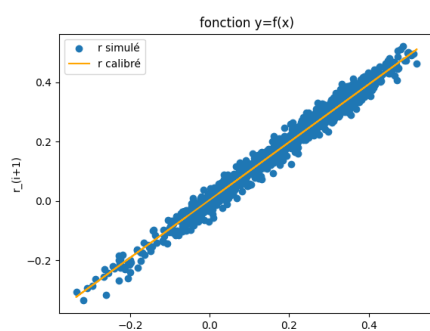
DateHisto()
```

Graphiques :



Nous allons ajuster le modèle de Vasicek aux données historiques par trois méthodes différentes.

3.1 - Régression par Levenberg-Marquardt



```
Beta1 = 0.9748210798278321
Beta2 = 0.0036610771510545786
variance D² = 3.117105834931614e-05
D = 0.00558310472311922
Gamma = 5.100266538561842
Sigma = 0.0799658910021759
Eta = 0.7415913454961027
```


Interprétation :

L'algorithme de Levenberg-Marquardt est utilisé pour l'optimisation non linéaire et dans le but de minimiser l'erreur entre les prédictions du modèle et les données observées. Les paramètres estimés permettent d'adapter le modèle au comportement des taux historiques.

3.2 - Régression par la méthode analytique

Code :

```
def Simuvasicek(N, gamma, dt, eta, sigma):
    tab_r = [0.023]

    for i in range(N-1):
        # Calcul de r
        r_next = tab_r[-1]*np.exp(-gamma*dt) + eta/gamma*(1-np.exp(-gamma*dt)) + sigma*np.sqrt((1-np.exp(-2*gamma*dt))/(2*gamma)) * np.random.randn()
        tab_r.append(r_next)

    return tab_r[:-1], tab_r[1:]

# Paramètres
dt = 0.01
r0 = 0.023
N=1000
T=5
dt=T/N
eta=0.6
sigma=0.08
gamma=4

tab_r_i, tab_r_il = Simuvasicek(N, gamma, dt, eta, sigma)

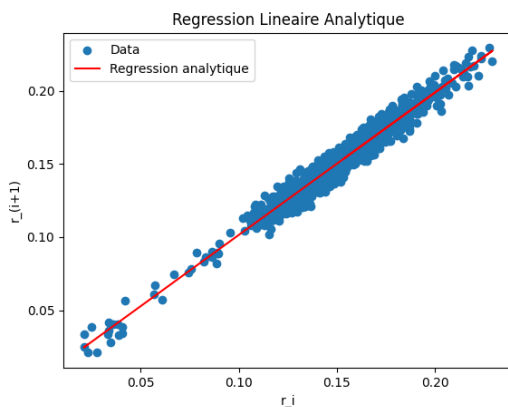
# Assurer que tab_r_i est un tableau NumPy
tab_r_i = np.array(tab_r_i)

# Matrice
X = np.column_stack((np.ones_like(tab_r_i), tab_r_i))

# Calculer les coefficients de régression analytiquement
beta_analytical = np.linalg.inv(X.T @ X) @ X.T @ tab_r_il

b_analytical, a_analytical = beta_analytical
print(f"Estimation analytique de a: {a_analytical}")
print(f"Estimation analytique de b: {b_analytical}")

# Visualiser les résultats
plt.scatter(tab_r_i, tab_r_il, label='Data')
plt.plot(tab_r_i, b_analytical + a_analytical * tab_r_i, color='red', label='Regression analytique')
plt.xlabel('r_i')
plt.ylabel('r_{i+1}')
plt.title('Regression Lineaire Analytique')
plt.legend()
plt.show()
```

Graphique :

```
Estimation analytique de a: 0.9734284679438124
Estimation analytique de b: 0.004126058794283718
```

Interprétation :

Appelée également “méthode des moindres carrés”, la régression linéaire par la méthode analytique est utilisée pour estimer les coefficients d’un modèle linéaire. Nous minimisons la somme des carrés des écarts entre les valeurs observées et les valeurs prédites par le modèle. Les coefficients calculés permettent de modéliser la relation linéaire entre les variables dépendantes et indépendantes.